

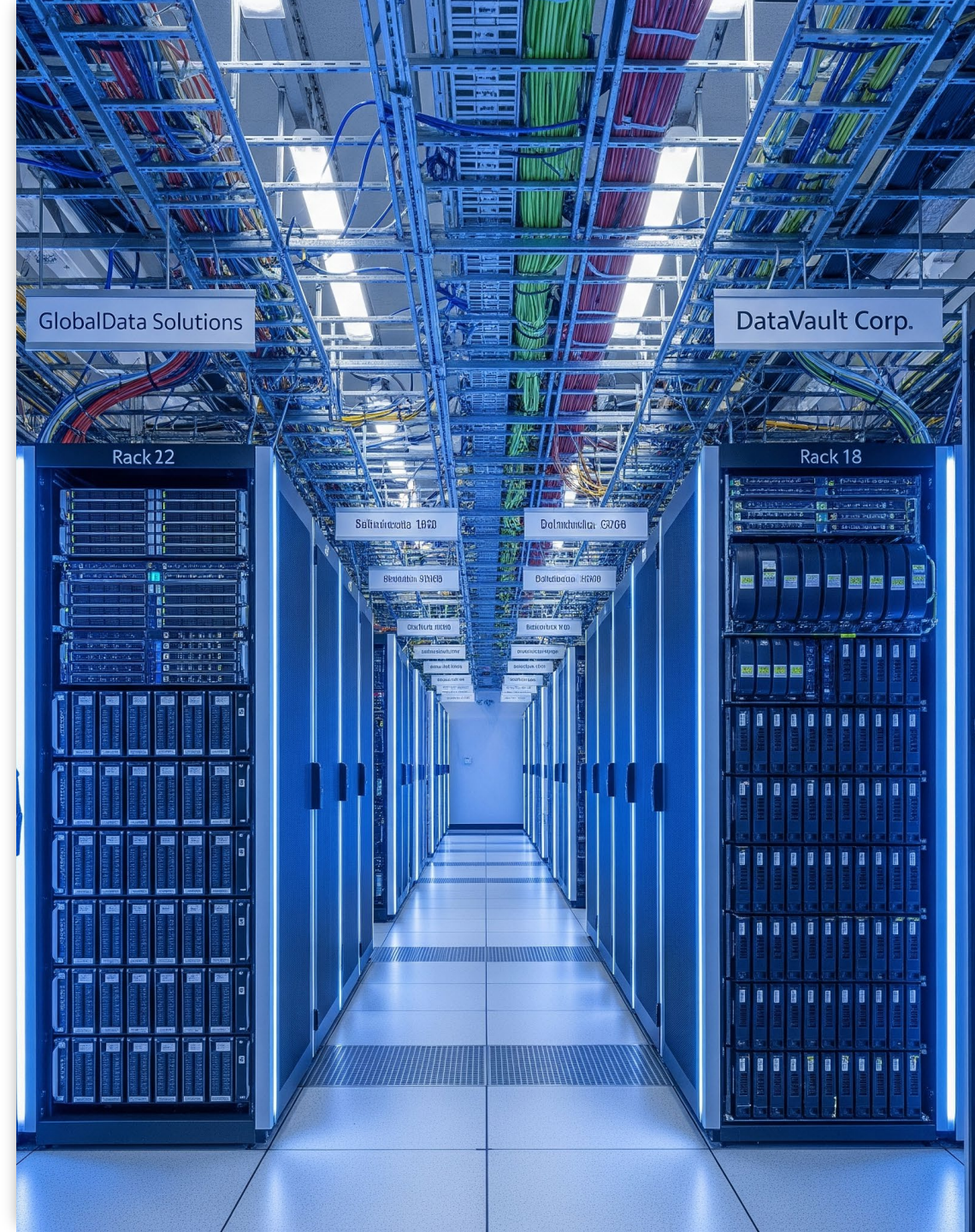


CTD Python Essentials

Week 8: Introduction to databases and SQL

Databases

- An efficient system for storing, searching and retrieving large quantities of data.
- Table like structured data accessed by a **key** which selects a **row** or **value**
- **Rows** in a tables are referred to as **records** for many types of database
 - A uniform set of **fields** or **columns** with specific datatypes
- Searching for data is referred to as a **Query**
- **Relational** database (**RDBMS**)
 - Tables accessed using a declarative language
 - **Relational Database Management System**





Database history

- Paper ledgers with structured rows and columns have been around for centuries
- Automation started with the punched card tabulating systems
 - Invented by Herman Hollerith in the 1880's and used for the 1890 US census
 - Introduced the idea of structured, tabular data with records
 - Company was eventually renamed IBM (International Business Machines)
- Further invention by IBM
 - IMS (Information Management System) for NASA in the 1960s Apollo program
 - Charles Bachman's IDS (Integrated Data Store) – Turing award winner 1973
 - Edgar F. Codd - "A Relational Model of Data for Large Shared Data Banks"
- System R at IBM (1970s)
 - SEQUEL later SQL
- Ingres at UC Berkeley - Michael Stonebraker
- Oracle – first commercial database (outside IBM)

SQL Databases

- **SQL** – **S**tructured **Q**uery **L**anguage
 - Created as part of IBMs System R in 1973
 - Became a standard in the 1980s (**ISO / IEC 9075**)
- A few important examples among many implementations
 - Free: **MySQL**, **PostgreSQL**, **SQLite**
 - Commercial: **Microsoft SQL Server**, **Oracle Database**, **IBM Db2**
- So popular and pervasive that everything else is referred to as **noSQL**
 - Less structured data often in a **Key / Value** store
 - Examples like **MongoDB** and **Cassandra**
 - *Graph databases* like **Neo4j**

SQL Key Concepts

- **Schema** – The definition of a database, specifying **tables**, **column** types and often valid **column** values
- **Table** – spreadsheet like structure with rows and columns; a database usually contains multiple **tables**
- **Record** – A row in a **table**
- **Columns** – uniform fields with specific datatypes in each **record**
- **Primary key** – integer used to address database records in a specific **table**
- **Foreign key** – A key stored in a **record column** which points to a **record** in a different **table**
- **Association** – Operation which combines access to multiple **tables** using **foreign keys**
- **Transaction** – Atomic operations on a database to avoid race conditions and ensure consistency
- **PRAGMA** – An implementation specific command which is outside of the **SQL** language

SQLite

- We will focus on SQLite for our lessons
- Simple efficient single file storage
- Pervasive application support
- Good programming support in python and many other languages
- Simple, limited set of datatypes (TEXT, NUMERIC, INTEGER, REAL, or BLOB)
- Many databases have a client/server architecture
 - Login, validation, admin, network connections all complicate setup
 - Sqlite uses a single file and avoids the complexity of client/server architecture
 - It is common to bring up an application using SQLite and then switch to a client/server database for distributed deployment
- SQLite was invented for use in software aboard a naval vessel
 - Wanted to avoid complexities in setup and deployment
 - [Origins of SQLite](#)

SQLite and Python

- Uses the sqlite3 module
- Connections are used to open a database for access
 - Connections must be **closed** when complete
 - Any changes during a connection must be **committed**
 - As with file operations, the **with** construct is usually used for connections since exiting the block closes the connection
- A [cursor](#) is a control structure which is used to interact with a database through a connection. Common cursor methods:
 - **execute()** – run a single SQL statement
 - **fetchall()** – Fetch all of the remaining rows of a query operation

```
import sqlite3

# Connect to the database
with sqlite3.connect("example.db") as conn:
    cursor = conn.cursor()

    # Create a table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL
        )
    """)

    # Insert data
    cursor.execute("INSERT INTO users (name) VALUES (?)", ("Alice",))

    # Query data
    cursor.execute("SELECT * FROM users")
    rows = cursor.fetchall()
    for row in rows:
        print(row)
```

CRUD in SQL

- An acronym for operations which almost all database applications need
 - **C**reate, **R**ead, **U**psdate, **D**eleate
 - Examples follow
- Operation on tables
 - Create a table: CREATE TABLE
 - Delete a table: DROP TABLE
- Operations on records
 - Create - INSERT
 - Read – SELECT, JOIN
 - Update – UPDATE
 - Delete - DELETE



Creating tables

```
# Create employees table
cursor.execute('''
CREATE TABLE IF NOT EXISTS employees (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    department TEXT NOT NULL,
    salary REAL NOT NULL,
    hire_date DATE,
    email TEXT UNIQUE
)
''')

# Create departments table for JOIN examples
cursor.execute('''
CREATE TABLE IF NOT EXISTS departments (
    dept_id INTEGER PRIMARY KEY,
    dept_name TEXT NOT NULL,
    manager_name TEXT,
    budget REAL
)
''')
```

```
# Create projects table for complex queries
cursor.execute('''
CREATE TABLE IF NOT EXISTS projects (
    project_id INTEGER PRIMARY KEY,
    project_name TEXT NOT NULL,
    employee_id INTEGER,
    start_date DATE,
    end_date DATE,
    status TEXT CHECK(status IN ('active', 'completed',
| | | | | | | | 'on_hold')),
    FOREIGN KEY (employee_id) REFERENCES employees(id)
)
''')

print("Tables created successfully")
conn.commit()
```

Removing tables

```
# First, let's show what tables exist
cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
tables_before = cursor.fetchall()
print("Tables before DROP:")
for table in tables_before:
    print(f" {table[0]}")

# Drop the projects table (careful with foreign keys!)
cursor.execute("DROP TABLE IF EXISTS projects")
print("Dropped projects table")

# Drop departments table
cursor.execute("DROP TABLE IF EXISTS departments")
print("Dropped departments table")
```

```
# Show remaining tables
cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
tables_after = cursor.fetchall()
print("\nTables after DROP:")
for table in tables_after:
    print(f" {table[0]}")

# Clean up - drop remaining table
cursor.execute("DROP TABLE IF EXISTS employees")
print("Dropped employees table")

print("\nFinal table count:")
cursor.execute("SELECT COUNT(*) FROM sqlite_master WHERE type='table'")
final_count = cursor.fetchone()[0]
print(f" Remaining tables: {final_count}")
```

```
# Insert into departments
departments_data = [
    (1, 'Engineering', 'Carol Davis', 500000),
    (2, 'Marketing', 'Sarah Wilson', 200000),
    (3, 'Sales', 'Mike Johnson', 300000),
    (4, 'HR', 'Linda Brown', 150000)
]

cursor.executemany('''
INSERT INTO departments (dept_id, dept_name, manager_name, budget)
VALUES (?, ?, ?, ?)
''', departments_data)
```

```
# Insert into employees
employees_data = [
    ('Alice Johnson', 'Engineering', 75000, '2022-01-15', 'alice@company.com'),
    ('Bob Smith', 'Marketing', 62000, '2021-03-20', 'bob@company.com'),
    ('Carol Davis', 'Engineering', 82000, '2020-07-10', 'carol@company.com'),
    ('David Wilson', 'Sales', 58000, '2023-02-01', 'david@company.com'),
    ('Eve Brown', 'Engineering', 78000, '2022-11-05', 'eve@company.com'),
    ('Frank Miller', 'HR', 65000, '2021-09-15', 'frank@company.com')
]

cursor.executemany('''
INSERT INTO employees (name, department, salary, hire_date, email)
VALUES (?, ?, ?, ?, ?)
''', employees_data)
```

```
# Insert into projects
projects_data = [
    ('Website Redesign', 1, '2023-01-01', '2023-06-30', 'completed'),
    ('Mobile App', 3, '2023-03-15', '2023-12-31', 'active'),
    ('Database Migration', 1, '2023-02-01', '2023-04-30', 'completed'),
    ('Marketing Campaign', 2, '2023-05-01', '2023-07-31', 'active'),
    ('HR System Upgrade', 6, '2023-04-01', None, 'on_hold')
]

cursor.executemany('''
INSERT INTO projects (project_name, employee_id, start_date, end_date, status)
VALUES (?, ?, ?, ?, ?)
''', projects_data)
```

Creating records

SELECT - finding records (Read from CRUD)

```
print("1. Basic SELECT - All employees:")
cursor.execute("SELECT * FROM employees")
results = cursor.fetchall()
for row in results:
    print(f" {row}")

print("\n2. SELECT specific columns:")
cursor.execute("SELECT name, department, salary FROM employees")
results = cursor.fetchall()
for row in results:
    print(f" {row}")

print("\n3. SELECT with WHERE clause:")
cursor.execute("SELECT name, salary FROM employees WHERE salary > 70000")
results = cursor.fetchall()
for row in results:
    print(f" {row}")
```

More examples of SELECT

```
print("\n4. SELECT with ORDER BY:")
cursor.execute("SELECT name, salary FROM employees ORDER BY salary DESC")
results = cursor.fetchall()
for row in results:
    print(f" {row}")

print("\n5. SELECT with LIMIT:")
cursor.execute("SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3")
results = cursor.fetchall()
for row in results:
    print(f" {row}")
```

Updating records

```
print("Before UPDATE - Eve Brown's salary:")
cursor.execute("SELECT name, salary FROM employees WHERE name = 'Eve Brown'")
result = cursor.fetchone()
print(f" {result}")

# Update single record
cursor.execute('''
UPDATE employees
SET salary = 85000
WHERE name = 'Eve Brown'
''')

print(f"Updated {cursor.rowcount} record(s)")

print("After UPDATE - Eve Brown's salary:")
cursor.execute("SELECT name, salary FROM employees WHERE name = 'Eve Brown'")
result = cursor.fetchone()
print(f" {result}")
```

```
# Update multiple records
cursor.execute('''
UPDATE employees
SET salary = salary * 1.05
WHERE department = 'Engineering'
''')

print(f"Gave 5% raise to {cursor.rowcount} Engineering employees")
conn.commit()
```

Deleting records

```
print("Before DELETE - All employees count:")
cursor.execute("SELECT COUNT(*) FROM employees")
count_before = cursor.fetchone()[0]
print(f" Total employees: {count_before}")

# Insert a temporary record to delete
cursor.execute('''
INSERT INTO employees (name, department, salary, hire_date, email)
VALUES ('Temp Employee', 'Temp', 50000, '2023-01-01', 'temp@company.com')
''')

print("After INSERT - Employee count:")
cursor.execute("SELECT COUNT(*) FROM employees")
count_after_insert = cursor.fetchone()[0]
print(f" Total employees: {count_after_insert}")

# Delete the temporary record
cursor.execute("DELETE FROM employees WHERE name = 'Temp Employee'")
print(f"Deleted {cursor.rowcount} record(s)")

print("After DELETE - Final employee count:")
cursor.execute("SELECT COUNT(*) FROM employees")
count_final = cursor.fetchone()[0]
print(f" Total employees: {count_final}")
conn.commit()
```


Joins

- Combining tables to reveal deeper insights
- INNER
 - Rows with matching foreign keys
- LEFT
 - Left rows, matching right rows
- RIGHT
 - Right rows and matching left rows
- Multiple – JOINS can be chained together when more than two tables are involved

```
print("1. INNER JOIN - Employees with their projects:")
cursor.execute('''
SELECT e.name, e.department, p.project_name, p.status
FROM employees e
INNER JOIN projects p ON e.id = p.employee_id
ORDER BY e.name
''')
results = cursor.fetchall()
for row in results:
    print(f" {row}")

print("\n2. LEFT JOIN - All employees and their projects (if any):")
cursor.execute('''
SELECT e.name, e.department, p.project_name, p.status
FROM employees e
LEFT JOIN projects p ON e.id = p.employee_id
ORDER BY e.name
''')
results = cursor.fetchall()
for row in results:
    print(f" {row}")

print("\n3. Multiple JOINS - Complex relationship:")
cursor.execute('''
SELECT e.name as employee, d.dept_name as department,
       d.manager_name as manager, p.project_name as project
FROM employees e
JOIN departments d ON e.department = d.dept_name
LEFT JOIN projects p ON e.id = p.employee_id
ORDER BY d.dept_name, e.name
''')
results = cursor.fetchall()
for row in results:
    print(f" {row}")
```

JOIN results

```
1. INNER JOIN - Employees with their projects:
('Alice Johnson', 'Engineering', 'Website Redesign', 'completed')
('Alice Johnson', 'Engineering', 'Database Migration', 'completed')
('Bob Smith', 'Marketing', 'Marketing Campaign', 'active')
('Carol Davis', 'Engineering', 'Mobile App', 'active')
('Frank Miller', 'HR', 'HR System Upgrade', 'on_hold')

2. LEFT JOIN - All employees and their projects (if any):
('Alice Johnson', 'Engineering', 'Database Migration', 'completed')
('Alice Johnson', 'Engineering', 'Website Redesign', 'completed')
('Bob Smith', 'Marketing', 'Marketing Campaign', 'active')
('Carol Davis', 'Engineering', 'Mobile App', 'active')
('David Wilson', 'Sales', None, None)
('Eve Brown', 'Engineering', None, None)
('Frank Miller', 'HR', 'HR System Upgrade', 'on_hold')

3. Multiple JOINS - Complex relationship:
('Alice Johnson', 'Engineering', 'Carol Davis', 'Database Migration')
('Alice Johnson', 'Engineering', 'Carol Davis', 'Website Redesign')
('Carol Davis', 'Engineering', 'Carol Davis', 'Mobile App')
('Eve Brown', 'Engineering', 'Carol Davis', None)
('Frank Miller', 'HR', 'Linda Brown', 'HR System Upgrade')
('Bob Smith', 'Marketing', 'Sarah Wilson', 'Marketing Campaign')
('David Wilson', 'Sales', 'Mike Johnson', None)
```

Complex queries

```
print("Advanced query: Engineering employees with active projects, salary analysis")
cursor.execute('''
SELECT
    e.name,
    e.salary,
    e.hire_date,
    COUNT(p.project_id) as active_projects,
    CASE
        WHEN e.salary > 80000 THEN 'Senior'
        WHEN e.salary > 70000 THEN 'Mid-level'
        ELSE 'Junior'
    END as salary_tier,
    ROUND((e.salary - AVG(e.salary) OVER()) / 1000, 2) as salary_diff_from_avg,
    RANK() OVER (ORDER BY e.salary DESC) as salary_rank,
    julianday('now') - julianday(e.hire_date) as days_employed
FROM employees e
LEFT JOIN projects p ON e.id = p.employee_id AND p.status = 'active'
WHERE e.department = 'Engineering'
GROUP BY e.id, e.name, e.salary, e.hire_date
HAVING COUNT(p.project_id) >= 0 -- Include all, even those with no active projects
ORDER BY e.salary DESC, active_projects DESC
''')

results = cursor.fetchall()
print("Results (Name, Salary, Hire Date, Active Projects, Tier, Salary Diff, Rank, Days Employed):")
for row in results:
    print(f" {row}")
```

Complex Query Results

Advanced query: Engineering employees with active projects, salary analysis

Results (Name, Salary, Hire Date, Active Projects, Tier, Salary Diff, Rank, Days Employed):

('Eve Brown', 89250.0, '2022-11-05', 0, 'Senior', 4.55, 1, 1025.0457354513928)

('Carol Davis', 86100.0, '2020-07-10', 1, 'Senior', 1.4, 2, 1873.0457354513928)

('Alice Johnson', 78750.0, '2022-01-15', 0, 'Mid-level', -5.95, 3, 1319.0457354513928)

=====

Complex query breakdown:

- SELECT: Multiple columns with calculations and window functions
- CASE: Conditional logic for salary tiers
- COUNT: Aggregate function with GROUP BY
- AVG() OVER(): Window function for average salary
- RANK() OVER(): Window function for ranking
- julianday(): Date calculations
- LEFT JOIN: Include all employees even without active projects
- WHERE: Filter by department
- GROUP BY: Group results for aggregation
- HAVING: Filter grouped results
- ORDER BY: Sort by multiple columns

SQL and Pandas

```
conn = sqlite3.connect(':memory:')
conn.execute('CREATE TABLE employees (name TEXT, department TEXT, salary REAL)')
employees = [('Alice', 'Engineering', 75000), ('Bob', 'Marketing', 62000),
             ('Carol', 'Engineering', 82000), ('David', 'Sales', 58000)]
conn.executemany('INSERT INTO employees VALUES (?, ?, ?)', employees)

# 1. SQL to DataFrame
df = pd.read_sql_query("SELECT * FROM employees", conn)
print("SQL query to a pandas DataFrame:")
print(df)

# 2. DataFrame to SQL
new_data = pd.DataFrame({'name': ['Eve'], 'department': ['HR'], 'salary': [65000]})
new_data.to_sql('employees', conn, if_exists='append', index=False)
print(f"\n✓ Added {len(new_data)} row via pandas")

# 3. Combined analysis
result = pd.read_sql_query("SELECT department, AVG(salary) as avg_salary FROM employees GROUP BY department", conn)
print("\nDepartment salary averages:")
print(result)

conn.close()
```

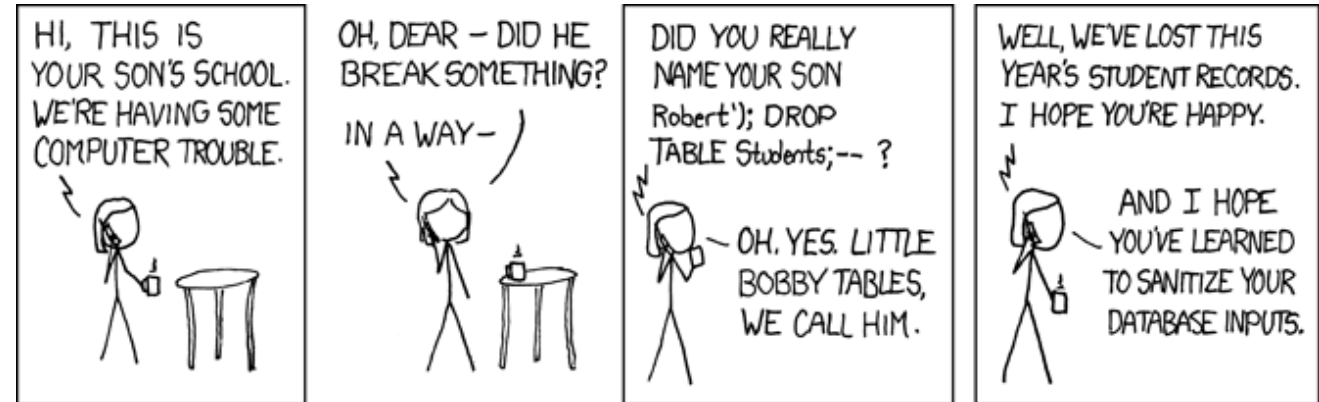
Performance Considerations

- SQL implementations are highly optimized and efficient
- Operations are optimized before they are executed
- When in doubt, let SQL do the work
- Anything you build in python is likely to be much slower and use more memory



Gotchas

- **A couple of important design considerations to look out for**
 - Not a comprehensive list of gotchas!
- **Define the schema in one place**
 - Schema-specific code in multiple places can be very fragile
 - ORMs (Object Relational Mapping) like [sqlalchemy](#) help with this
 - JSON can be used to create a central schema definition, even including valid values for a given column
- **Prevent Injection attacks**
 - Don't allow user inputs to be executed directly as SQL
 - Use parameters rather than interpolation
 - That's why the examples all look like:
 - ...WHERE department_id = ?"; (department_id,)



From <https://xkcd.com/>

Q&A and Demo

Amateur radio contact database:

[demo project link](#)

Gist with code examples:

[CTD Python 100 SQL Week 1](#)

